

Blind Dereverberation Exploiting the Null of Sylvester Matrix with Group Sparse Regularization in the STFT domain

打撃音的な位置で周波数方向に位相が一括でリセットされる性質を使いたかった実験

```
clear
```

settings

```
% DGT tool
F = DGTtool("windowLength", 512, "windowShift", 128);

% audio data loading
num_piece = 2;
for i=1:num_piece
    audiofilename = "Goldberg_clav_mono_ds_%d";
    %audiofilename = "9w3r7y_01_%d";
    [temp, fs] = audioread("input/" + sprintf(audiofilename,i)+".wav");
    ss{i} = temp(:,1);
end
s = [ss{1}; ss{2}];
num_mic = 1;

% ir data loading
[ir, ~] = audioread("BFTLSPACELIBRARY_ChamberAL_11025.wav");
%[ir, fs] = audioread("logic03sWoodenBooth8000.wav");
%[ir{2}, fs(2)] = audioread("../data/BF TL SPACE LIBRARY/Drumbrella/Drumbrella
5'.R.wav");

% ADMM settings
rho = 1.0; % ADMM のペナルティパラメータ (調整用、まずは1.0等から)
alpha = 0.01; % L1 正則化の強さ (大きくするほど音漏れがゼロに潰れる)

% HPSS setting
params = getParamsHPSS(F, fs);
```

Malloc

```
rt = zeros(1,1); % reverb time in sec
obs = cell(1,num_piece); % obs
X = cell(1, num_piece); % STFTs of obs. signal
S = cell(1, num_piece); % STFTs of dry signal
N = zeros(1, num_piece); % length of observation
Lir = zeros(1, 1); % ir length = d + 1
l = zeros(1, num_piece); % length of deconvoluted signal
```

resample and shortening

```

% calculate the reverberation time (RT60)
rt = reverb_time(ir, fs);
ir = ir(1:floor(fs*rt)); ir = ir / norm(ir);
IR = F(ir);
Lir = size(IR, 2);

for j = 1:num_piece
    obs{j} = conv(ss{j}, ir);
    X{j} = F(obs{j});
    N(j) = size(X{j}, 2);
    S{j} = F(ss{j});
end
obsCat = [obs{1}; obs{2}];
D = Lir - 1;
padding = -2;
D = D + padding;
Lir = Lir + padding;

```

deconvolution in TF domain

malloc

```

Fq = size(X{1}, 1);
for i = 1:2, l(i) = N(1, i) - Lir(1) + 1; end
RET1 = zeros(Fq, l(1)); RET2 = zeros(Fq, l(2));
RET1W = zeros(Fq, l(1)); RET2W = zeros(Fq, l(2));
RET_slra = zeros(Fq, sum(l));
% ratio of minimum and the 2nd minimum singular value
singValRatio = zeros(Fq, 1);

% frequency-wise

```

時間領域で観測 (x) がクリーン信号 (s) と IR (h) の畳み込みで表されているとする:

$$\text{Time domain: } \mathbf{x} = \mathbf{s} * \mathbf{h}.$$

それらの DGT ($\mathbf{X}_f, \mathbf{S}_f, \mathbf{H}_f$) に対して、周波数 f のサブバンドで畳み込みが近似的に成り立っているという立場をとる.

$$\text{TF domain } \mathbf{X}_f \approx \mathbf{S}_f * \mathbf{H}_f$$

今、独立な二つのクリーン信号 s^1, s^2 があって、同一の IR が畳み込まれたものを観測しているとする:

$$\text{Time domain : } \mathbf{x}^1 = \mathbf{s}^1 * \mathbf{h}, \mathbf{x}^2 = \mathbf{s}^2 * \mathbf{h}.$$

TF 領域では、それに対応して

$$\text{TF domain : } \mathbf{X}_f^1 = \mathbf{S}_f^1 * \mathbf{H}_f, \mathbf{X}_f^2 = \mathbf{S}_f^2 * \mathbf{H}_f$$

がある。式をいじって、次を得る:

$$\text{TF domain : } \mathbf{X}_f^1 * \mathbf{S}_f^2 = \mathbf{X}_f^2 * \mathbf{S}_f^1, = \mathbf{S}_f^1 * \mathbf{S}_f^2 * \mathbf{H}_f.$$

一つ目の等号の部分から

$$\text{TF domain : } \mathbf{X}_f^1 * \mathbf{S}_f^2 - \mathbf{X}_f^2 * \mathbf{S}_f^1 = \mathbf{0}$$

これを畳み込み行列を使って表すと,

TF domain :

$$\text{Conv}(\mathbf{X}_f^1) \mathbf{S}_f^2 - \text{Conv}(\mathbf{X}_f^2) \mathbf{S}_f^1 = \mathbf{0}$$

↓

$$\begin{bmatrix} \text{Conv}(\mathbf{X}_f^2) & \text{Conv}(-\mathbf{X}_f^1) \end{bmatrix} \begin{bmatrix} \mathbf{S}_f^1 \\ \mathbf{S}_f^2 \end{bmatrix} = \mathbf{0}$$

(最後に出てきた式の係数行列は Sylvester 行列と名前がついている)

$$\text{Sylvester} \stackrel{\text{def}}{=} \begin{bmatrix} \text{Conv}(\mathbf{X}_f^2) & \text{Conv}(-\mathbf{X}_f^1) \end{bmatrix}$$

Sylvester 行列の零空間の基底ベクトルとしてクリーン信号 $\mathbf{S}_f^1$, $\mathbf{S}_f^2$ を獲得する.

TF 領域での「近似」畳み込みモデルを使っているので, 上記の Sylvester 行列はフルランク行列になっている,

つまり零空間はないのだが, その代わりに最小特異値に対応する右特異ベクトルを使用してクリーン信号の復元を試みる.

```
parfor f = 1:Fq
    temp = X{1, 1}; X1f = temp(f, :)' ;
    temp = X{1, 2}; X2f = temp(f, :)' ;
    [v1, v2] = sylDerev121(X1f, X2f, 1, rho, alpha);
    RET1(f,:) = v1'; RET2(f,:) = v2';
    % RET1W(f,:) = Vf_w(1:l(1))'; RET2W(f,:) = Vf_w(l(1)+1:end)';
end
```

周波数サブバンドごとに入り込んでランダムな位相遅延の修復

Motivation

畳み込みにおいて, スカラの乗算は畳み込み演算の前後どちらに入れ替えても結合則及び可換則が成立する.

$$\mathbf{x}_f \approx \mathbf{s}_f * \hat{\mathbf{h}}_f = (c_f \mathbf{s}_f) * \left(\frac{1}{c_f} \hat{\mathbf{h}}_f \right)$$

これは、オールパスフィルタがかかっているのにとれている。この式からスカラ乗算の不定性を取り除き, 真のモデル $\mathbf{x}_f \approx \mathbf{s}_f * \hat{\mathbf{h}}_f$ と一致させるためには, サブバンド IR ベクトル $\hat{\mathbf{h}}_f$ に対応する時間領域の RIR に制約 (直接音は周波数に関係せず全てのサブバンドで同じ時

刻に到達するなどの、現実的に満たすべきものなどが挙げられる) をかけ、制約を満たす位相回転 c_f を求める必要がある。位相回転は、時間領域の逆畳み込みで IR を推定し、その位相から計算する。

時間領域での逆畳み込み

時間遅延の統一：各サブバンド f で独立して逆畳み込みを解いた場合、得られるサブバンド IR $\hat{\mathbf{h}}_f$ は、その周波数ビンの中で最も辻褃が合うように直接音の到来時刻を推定して、時間遅延をフィットしてしまう。しかし、現実の設定ではすべての周波数成分が同時にマイクに到達するため、全帯域で共通の物理的な時間遅延を持つ。

- サブバンドで個別に解いた場合：サブバンドごとに直接音の立ち上がりタイミングがズレて同定される。
- 時間領域に戻して一括で解いた場合：サブバンドに関係なく全周波数共通で直接音の立ち上がり時刻が強制的に固定され、周波数間で時間アライメントが揃う

IR のフーリエ変換 $H(f)$ とサブバンド IR $\mathbf{h}_f$ の関係

```
retMid{1} = F.pinv(RET1);    retMid{2} = F.pinv(RET2);
```

index computed (OLA)

```
%% --- [Step 1] HPSS による打撃音成分の抽出 ---
```

```
RET1mag = abs(RET1);
```

```
RET1magharm = movmedian(RET1mag, params.medtfllen, 2);
```

```
RET1magperc = movmedian(RET1mag, params.medffllen, 1);
```

```
totalMagnitudePerBin = RET1magharm + RET1magperc + eps; % ゼロ除算防止
```

```
percussiveMask = RET1magperc > (totalMagnitudePerBin * 0.5);
```

```
RET1perc = percussiveMask .* RET1;
```

```
figure, tiledlayout(3,1)
```

```
nexttile; imagesc(abs(flipud(RET1))); title("estimated amp")
```

```
nexttile; imagesc(abs(flipud(RET1perc))); title("detected percussive frame")
```

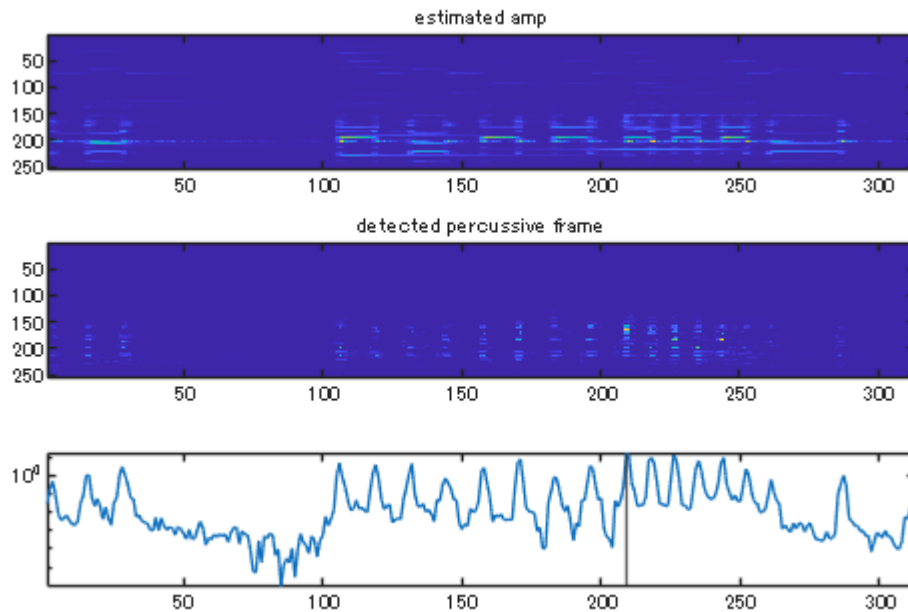
```
%% --- [Step 2] 最も強く打撃音が出ているフレームの特定 ---
```

```
percussive_energy_profile = sum(abs(RET1perc).^2, 1);
```

```
nexttile; semilogy(percussive_energy_profile);
```

```
[~, t_attack] = max(percussive_energy_profile); % locate the frame with maximum power
```

```
hold on; xline(t_attack); xlim([1, size(RET1, 2)])
```



```
% --- [Step 3] アンカーフレームの位相を基準にした位相回転角の推定 ---
% アンカーフレームにおける現在の（位相が狂った）複素スペクトル
theta_observed = angle(RET1(:, t_attack));
```

```
% 物理的拘束条件の適用
```

```
% 本来、理想的な打撃音の位相は全周波数で一律であるべき
```

```
% サブバンド間での整合性が取ればいいので、位相は0と仮定
```

```
% このとき観測された偏角 theta_observed がそのままサブバンドの位相回転
```

```
theta_error = theta_observed;
```

```
% --- [Step 4] 全時間フレームにわたる位相の一括アライメント ---
```

```
[num_freqs, num_frames] = size(RET1);
theta_error_matrix = repmat(theta_error, 1, num_frames);
phase_rectifier = exp(-1j * theta_error_matrix);
```

```
RET1_rectified = RET1 .* phase_rectifier;
```

```
RET2_rectified = RET2 .* phase_rectifier;
```

```
% ㊦ 最終的な残響除去音声の出力
```

```
ret{1} = F.pinv(RET1_rectified);
```

```
ret{2} = F.pinv(RET2_rectified);
```

Display

```
for i=1:2
    len_ss = length(ss{i});
    len_ret = length(ret{i});
```

```

if len_ss > len_ret
    ss{i} = ss{i}(1:len_ret);
    len_ss = len_ret;
end
[SDR,SIR,SAR,perm] = bss_eval_sources(obs{i}(1:len_ss)', ss{i}');
fprintf("obs: SDR %2.3f, SIR %2.3f, SAR %2.3f\n", SDR, SIR, SAR)

[SDR,SIR,SAR,perm] = bss_eval_sources(retMid{i}(1:len_ss)', ss{i}');
fprintf("ret(mid): SDR %2.3f, SIR %2.3f, SAR %2.3f\n", SDR, SIR, SAR)

[SDR,SIR,SAR,perm] = bss_eval_sources(ret{i}(1:len_ss)', ss{i}');
fprintf("ret: SDR %2.3f, SIR %2.3f, SAR %2.3f\n", SDR, SIR, SAR)

audiowrite("output/" + sprintf(audiofilename + "_wet.wav", i), obs{i}, fs)
audiowrite("output/" + sprintf(audiofilename + "_lra_l21_phased_pad=%d.wav", i,
padding), ret{i}, fs)
audiowrite("output/" + sprintf(audiofilename + "_lra_l21_pad=%d.wav", i,
padding), retMid{i}, fs)
end

```

```

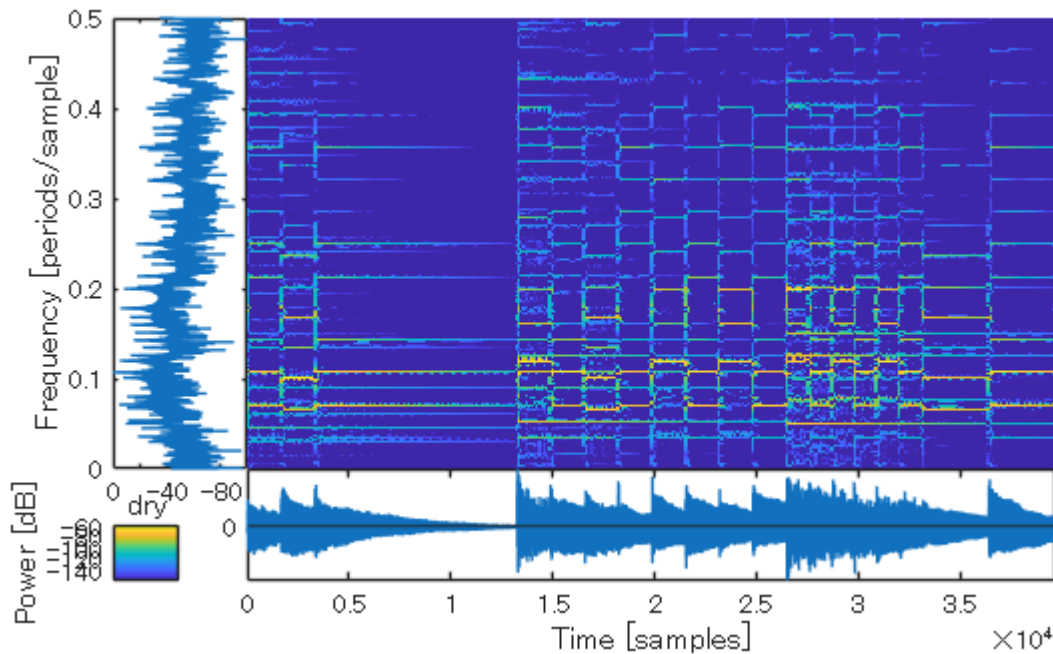
obs: SDR -2.082, SIR Inf, SAR -2.082
ret(mid): SDR 7.167, SIR Inf, SAR 7.167
ret: SDR 6.259, SIR Inf, SAR 6.259
obs: SDR -0.594, SIR Inf, SAR -0.594
ret(mid): SDR 6.784, SIR Inf, SAR 6.784
ret: SDR 6.353, SIR Inf, SAR 6.353

```

```

F.plotReassign(ss{1});title("dry");
saveas(gcf, "result/" + sprintf(audiofilename,1) + "_dry.png")

```

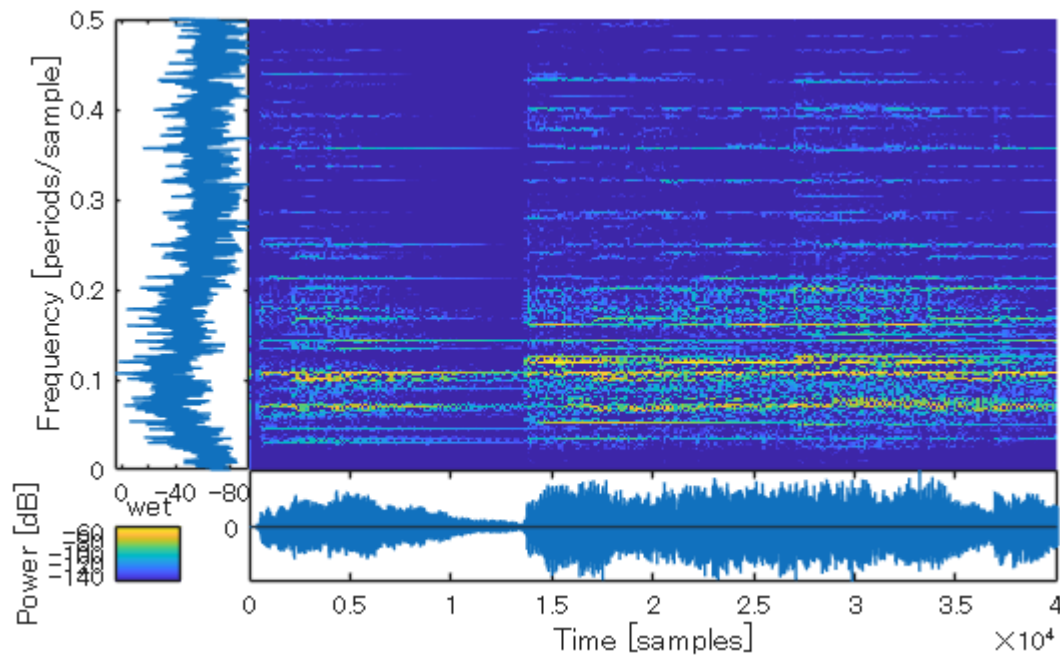


```

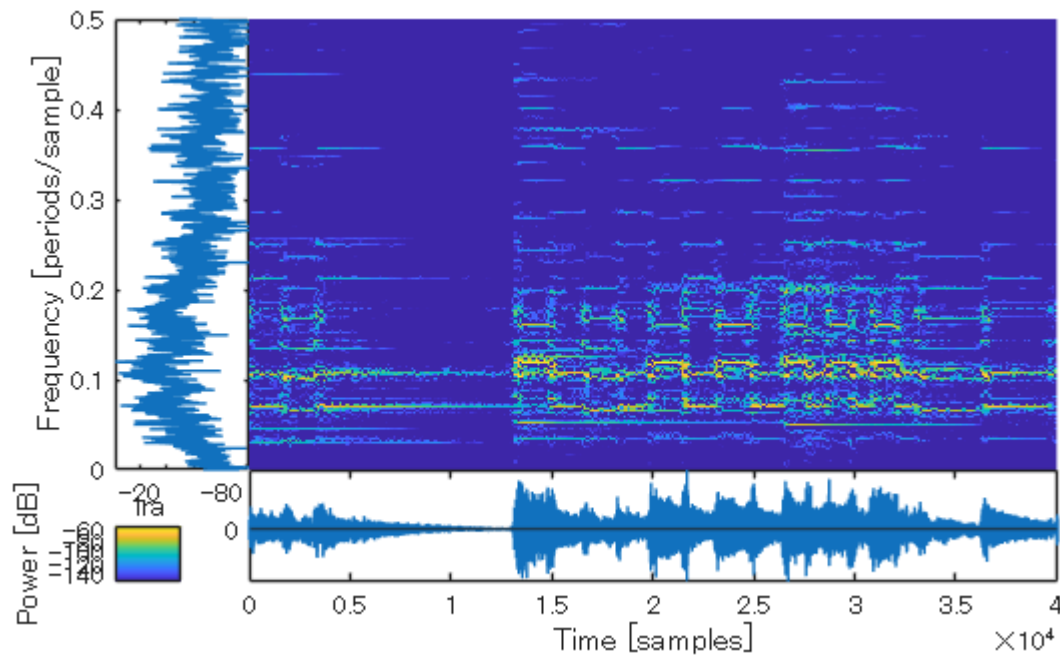
F.plotReassign(obs{1}(1:length(ret{1}))); title("wet");

```

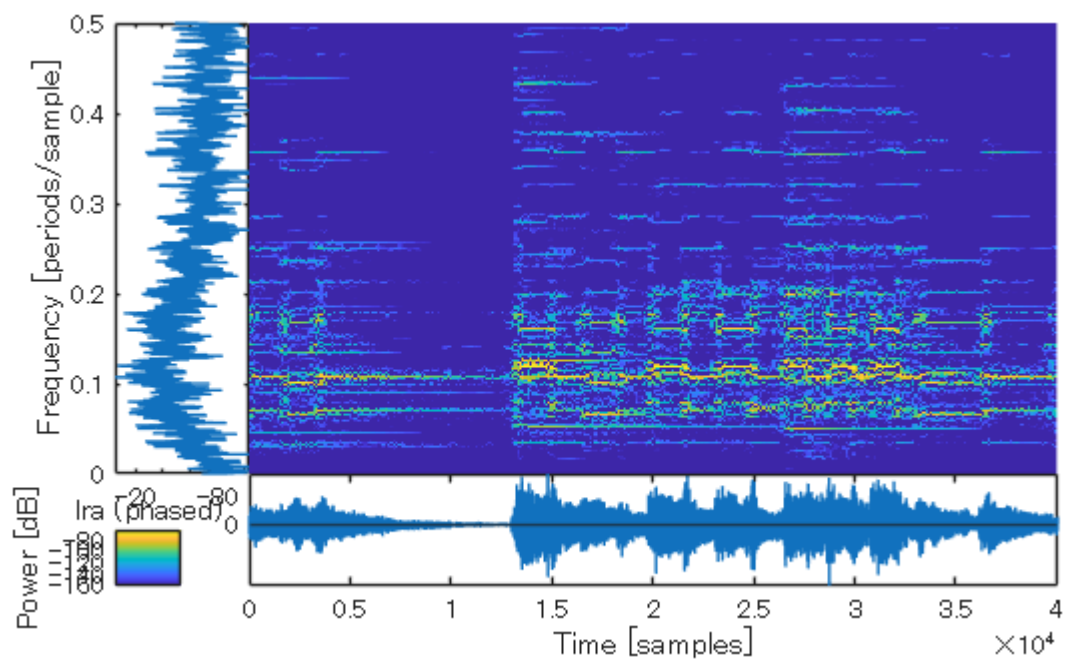
```
saveas(gcf, "result/" + sprintf(audiofilename,1) + "_wet.png")
```



```
F.plotReassign(retMid{1}); title("lra");
saveas(gcf, "result/" + sprintf(audiofilename + "_ret_pad=%d.png",1, padding) )
```



```
F.plotReassign(ret{1}); title("lra (phased)");
saveas(gcf, "result/" + sprintf(audiofilename + "_ret_phased_pad=%d.png",1,
padding) )
```



- どの結果も時間のマイナス方向に信号が漏れ出ているのがきになる.